

Chapter 6

Main Memory Scaling: Challenges and Solution Directions

Onur Mutlu, Carnegie Mellon University

Abstract

The memory system is a fundamental performance and energy bottleneck in almost all computing systems. Recent system design, application, and technology trends that require more capacity, bandwidth, efficiency, and predictability out of the memory system make it an even more important system bottleneck. At the same time, DRAM technology is experiencing difficult *technology scaling* challenges that make the maintenance and enhancement of its capacity, energy-efficiency, and reliability significantly more costly with conventional techniques.

In this chapter, after describing the demands and challenges faced by the memory system, we examine some promising research and design directions to overcome challenges posed by memory scaling. Specifically, we describe three major solution directions: 1) enabling new DRAM architectures, functions, interfaces, and better integration of the DRAM and the rest of the system (an approach we call *system-DRAM co-design*), 2) designing a memory system that employs emerging non-volatile memory technologies and takes advantage of multiple different technologies (i.e., *hybrid memory systems*), 3) providing predictable performance and QoS to applications sharing the memory system (i.e., *QoS-aware memory systems*). We also briefly describe our ongoing related work in combating scaling challenges of NAND flash memory.

6.1 Introduction

Main memory is a critical component of all computing systems, employed in server, embedded, desktop, mobile and sensor environments. Memory capacity, energy, cost, performance, and management algorithms must scale as we scale the size of the computing system in order to maintain performance growth and enable new applications. Unfortunately, such scaling has become difficult because recent trends in systems, applications, and technology greatly exacerbate the memory system bottleneck.

6.2 Trends: Systems, Applications, Technology

In particular, on the *systems/architecture front*, energy and power consumption have become key design limiters as the memory system continues to be responsible for a significant fraction of overall system energy/power [69]. More and increasingly heterogeneous processing cores and agents/clients are sharing the memory system [6, 21, 107, 45, 46, 36, 23], leading to increasing demand for memory capacity and bandwidth along with a relatively new demand for predictable performance and quality of service (QoS) from the memory system [81, 87, 106].

On the *applications front*, important applications are usually very data intensive and are becoming increasingly so [8], requiring both real-time and offline manipulation of great amounts of data. For example, next-generation genome sequencing technologies produce massive amounts of sequence data that overwhelms memory storage and bandwidth requirements of today's high-end desktop and laptop systems [109, 4, 115] yet researchers have the goal of enabling low-cost personalized medicine. Creation of new *killer applications* and usage models for computers likely depends on how well the memory system can support the efficient storage and manipulation of data in such data-intensive applications. In addition, there is an increasing trend towards consolidation of applications on a chip to improve efficiency, which leads to the sharing of the memory system across many heterogeneous applications with diverse performance requirements, exacerbating the aforementioned need for predictable performance guarantees from the memory system [106, 108].

On the *technology front*, two major trends profoundly affect memory systems. First, there is increasing difficulty scaling the well-established charge-based memory technologies, such as DRAM [77, 53, 40, 5, 62, 1] and flash memory [59, 76, 10, 11, 14], to smaller technology nodes. Such scaling has enabled memory systems with reasonable capacity and efficiency; lack of it will make it difficult to achieve high capacity and efficiency at low cost. Second, some emerging resistive memory technologies, such as phase change memory (PCM) [100, 113, 62, 63, 98], spin-transfer torque magnetic memory (STT-MRAM) [19, 60] or resistive RAM (RRAM) [114] appear more scalable, have latency and bandwidth characteristics much closer to DRAM than flash memory and hard disks, and are non-volatile with little idle power consumption. Such emerging technologies can enable new opportunities in system design, including, for example, the unification of memory and storage subsystems [80]. They have the potential to be employed as part of main memory, alongside or in place of less scalable and leaky DRAM, but they also have various shortcomings depending on the technology (e.g., some have cell endurance problems, some have very high write latency/power, some have low density) that need to be overcome or tolerated.

6.3 Requirements: Traditional and New

System architects and users have always wanted more from the memory system: high performance (ideally, zero latency and infinite bandwidth), infinite capacity, all at zero cost! The aforementioned trends do not only exacerbate and morph the above requirements, but also add some new requirements. We classify the requirements from the memory system into two categories: *exacerbated traditional requirements* and *(relatively) new requirements*.

The traditional requirements of performance, capacity, and cost are greatly exacerbated today due to increased pressure on the memory system, consolidation of multiple applications/agents sharing the memory system, and difficulties in DRAM technology and density scaling. In terms of *performance*, two aspects have changed. First, today's systems and applications not only require low latency and high bandwidth (as traditional memory systems have been optimized for), but they also require new techniques to manage and control memory interference between different cores, agents, and applications that share the memory system [81, 87, 106, 26, 108] in order to provide high system performance as well as predictable performance (or quality of service) to different applications [106]. Second, there is a need for increased memory bandwidth for many applications as the placement of more cores and agents on chip make the memory pin bandwidth an increasingly precious resource that determines system performance [41], especially for memory-bandwidth-intensive workloads, such as GPGPUs [48, 47], heterogeneous systems [6], and consolidated workloads [87, 44, 43]. In terms of *capacity*, the need for memory capacity is greatly increasing due to the placement of multiple data-intensive applications on the same chip and continued increase in the data sets of important applications. One recent work showed that given that the core count is increasing at a faster rate than DRAM capacity, the expected memory capacity per core is to drop by 30% every two years [70], an alarming trend since much of today's software innovations and features rely on increased memory capacity. In terms of *cost*, increasing difficulty in DRAM technology scaling poses a difficult challenge to building higher density (and, as a result, lower cost) main memory systems. Similarly, cost-effective options for providing high reliability and increasing memory bandwidth are needed to scale the systems proportionately with the reliability and data throughput needs of today's data-intensive applications. Hence, the three traditional requirements of performance, capacity, and cost have become exacerbated.

The relatively new requirements from the main memory system are threefold. First, *technology scalability*: there is a new need for finding a technology that is much more scalable than DRAM in terms of capacity, energy, and cost, as described earlier. As DRAM continued to scale well from the above-100-nm to 30-nm technology nodes, the need for finding a more scalable technology was not a prevalent problem. Today, with the significant circuit and device scaling challenges DRAM has been facing below the 30-nm node, it is. Second, there is a relatively new need for providing *performance predictability and QoS* in the shared main memory system. As single-core systems were dominant and memory bandwidth and capacity were much less of a shared resource in the past, the need for predictable perfor-

mance was much less apparent or prevalent [81]. Today, with increasingly more cores/agents on chip sharing the memory system and increasing amounts of workload consolidation, memory fairness, predictable memory performance, and techniques to mitigate memory interference have become first-class design constraints. Third, there is a great need for much higher *energy/power/bandwidth efficiency* in the design of the main memory system. Higher efficiency in terms of energy, power, and bandwidth enables the design of much more scalable systems where main memory is shared between many agents, and can enable new applications in almost all domains where computers are used. Arguably, this is not a new need today, but we believe it is another first-class design constraint that has not been as traditional as performance, capacity, and cost.

6.4 Solution Directions

As a result of these systems, applications, and technology trends and the resulting requirements, it is our position that researchers and designers need to fundamentally rethink the way we design memory systems today to 1) overcome scaling challenges with DRAM, 2) enable the use of emerging memory technologies, 3) design memory systems that provide predictable performance and quality of service to applications and users. The rest of this chapter describes our solution ideas in these three directions, with pointers to specific techniques when possible. Since scaling challenges themselves arise due to difficulties in enhancing memory components at *solely* one level of the computing stack (e.g., the device and/or circuit levels in case of DRAM scaling), we believe effective solutions to the above challenges will require cooperation across different layers of the computing stack, from algorithms to software to microarchitecture to devices, as well as between different components of the system, including processors, memory controllers, memory chips, and the storage subsystem. As much as possible, we will give examples of such solutions and directions.

6.5 Challenge 1: New DRAM Architectures

DRAM has been the choice technology for implementing main memory due to its relatively low latency and low cost. DRAM process technology scaling has for long enabled lower cost per unit area by enabling reductions in DRAM cell size. Unfortunately, further scaling of DRAM cells has become costly [5, 77, 53, 40, 62, 1] due to increased manufacturing complexity/cost, reduced cell reliability, and potentially increased cell leakage leading to high refresh rates. Several key issues to tackle include:

- 1) reducing the negative impact of refresh on energy, performance, QoS, and density scaling [71, 72, 17],

- 2) improving DRAM parallelism/bandwidth [57, 17], latency [68], and energy efficiency [57, 68, 71],
- 3) improving reliability of DRAM at low cost [90, 75, 58, 51],
- 4) reducing the significant amount of waste present in today's main memories in which much of the fetched/stored data can be unused due to coarse-granularity management [79, 117, 94, 95, 110],
- 5) minimizing data movement between DRAM and processing elements, which causes high latency, energy, and bandwidth consumption [102].

Traditionally, DRAM devices have been separated from the rest of the system with a rigid interface, and DRAM has been treated as a *passive* slave device that simply responds to the commands given to it by the memory controller. We believe the above key issues can be solved more easily if we rethink the DRAM architecture and functions, and redesign the interface such that DRAM, controllers, and processors closely cooperate. We call this high-level solution approach *system-DRAM co-design*. We believe key technology trends, e.g., the 3D stacking of memory and logic [74, 2, 111] and increasing cost of scaling DRAM solely via circuit-level approaches [77, 53, 40], enable such a co-design to become increasingly more feasible. We proceed to provide several examples from our recent research that tackle the problems of refresh, parallelism, latency, and energy efficiency.

6.5.1 Reducing Refresh Impact and DRAM Error Management

With higher DRAM capacity, more cells need to be refreshed at likely higher rates than today. Our recent work [71] indicates that refresh rate limits DRAM density scaling: a hypothetical 64Gb DRAM device would spend 46% of its time and 47% of all DRAM energy for refreshing its rows, as opposed to typical 4Gb devices of today that spend respectively 8% of the time and 15% of the DRAM energy on refresh (as shown in Figure 6.1). Today's DRAM devices refresh all rows at the same worst-case rate (e.g., every 64ms). However, only a small number of weak rows require a high refresh rate [54, 72, 51] (e.g., only ~1000 rows in 32GB DRAM require to be refreshed more frequently than every 256ms). Retention-Aware Intelligent DRAM Refresh (RAIDR) [71] exploits this observation: it groups DRAM rows into bins (implemented as Bloom filters [7] to minimize hardware overhead) based on the retention time of the weakest cell within each row. Each row is refreshed at a rate corresponding to its retention time bin. Since few rows need high refresh rate, one can use very few bins to achieve large reductions in refresh counts: our results show that RAIDR with three bins (1.25KB hardware cost) reduces refresh operations by ~75%, leading to significant improvements in system performance and energy efficiency as described by Liu et al. [71].

Like RAIDR, other approaches have also been proposed to take advantage of the retention time variation of cells across a DRAM chip. For example, some works proposed refreshing weak rows more frequently at a per-row granularity, others proposed not using memory rows with low retention times, and yet others suggested

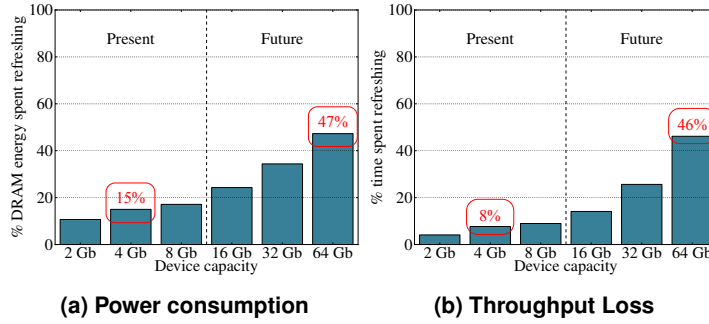


Fig. 6.1. Impact of refresh in current (DDR3) and projected DRAM devices. Reproduced from [71].

mapping critical data to cells with longer retention times such that critical data is not lost [112, 42, 73, 93, 52, 3] – see [71, 72] for a discussion of such techniques. Such approaches that exploit non-uniform retention times across DRAM require accurate retention time profiling mechanisms. Understanding of retention time as well as error behavior of DRAM devices is a critical research topic, which we believe can enable other mechanisms to tolerate refresh impact and errors at low cost. Liu et al. [72] provides an experimental characterization of retention times in modern DRAM devices to aid such understanding. Our initial results in that work, obtained via the characterization of 248 modern commodity DRAM chips from five different DRAM manufacturers, suggest that the retention time of cells in a modern device is largely affected by two phenomena: 1) Data Pattern Dependence, where the retention time of each DRAM cell is significantly affected by the data stored in other DRAM cells, 2) Variable Retention Time, where the retention time of a DRAM cell changes unpredictably over time. These two phenomena pose challenges against accurate and reliable determination of the retention time of DRAM cells, online or offline, and a promising area of future research is to devise techniques that can identify retention times of DRAM cells in the presence of data pattern dependence and variable retention time. Khan et al.’s recent work [51] provides more analysis of the effectiveness of conventional error mitigation mechanisms for DRAM retention failures and proposes *online retention time profiling* as a solution for identifying retention times of DRAM cells as a potentially promising approach in future DRAM systems.

Looking forward, we believe that increasing cooperation between the DRAM device and the DRAM controller as well as other parts of the system, including system software, is needed to communicate information about *weak* (or, unreliable) cells and the characteristics of different rows or physical memory regions from the device to the system. The system can then use this information to optimize data allocation and movement, refresh rate management, and error tolerance mechanisms. Low-cost error tolerance mechanisms are likely to be enabled more efficiently with such coordination between DRAM and the system. In fact, as DRAM technology

scales and error rates increase, it might become increasingly more difficult to maintain the common illusion that DRAM is a perfect, error-free storage device. DRAM may start looking increasingly like flash memory, where the memory controller manages errors such that an acceptable specified uncorrectable bit error rate is satisfied [10, 12]. We envision a *DRAM Translation Layer (DTL)*, not unlike the *Flash Translation Layer (FTL)* of today in spirit (which is decoupled from the processor and performs a wide variety of management functions for flash memory, including error correction, garbage collection, read/write scheduling, etc.), can enable better scaling of DRAM memory into the future by not only enabling easier error management but also opening up new opportunities to perform computation and mapping close to memory. This can become especially feasible in the presence of the trend of combining the DRAM controller and DRAM via 3D stacking. What should the interface be to such a layer and what should be performed in the DTL are promising areas of future research.

6.5.2 Improving DRAM Parallelism

A key limiter of DRAM parallelism is bank conflicts. Today, a bank is the finest-granularity independently accessible memory unit in DRAM. If two accesses go to the same bank, one has to *completely* wait for the other to finish before it can be started (see Figure 6.2). We have recently developed mechanisms, called SALP (subarray level parallelism) [57], that exploit the internal subarray structure of the DRAM bank (Figure 6.2) to *mostly* parallelize two requests that access the same DRAM bank. The key idea is to reduce the hardware sharing between DRAM subarrays such that accesses to the same bank but different subarrays can be initiated in a pipelined manner. This mechanism requires the exposure of the internal subarray structure of a DRAM bank to the controller and the design of the controller to take advantage of this structure. Our results show significant improvements in performance and energy efficiency of main memory due to parallelization of requests and improvement of row buffer hit rates (as row buffers of different subarrays can be kept active) at a low DRAM area overhead of 0.15%. Exploiting SALP achieves most of the benefits of increasing the number of banks at much lower area and power overhead than doing so. Exposing the subarray structure of DRAM to other parts of the system, e.g., to system software or memory allocators, can enable data placement and partitioning mechanisms that can improve performance and efficiency even further.

Note that other approaches to improving DRAM parallelism especially in the presence of refresh and long write latencies are also promising to investigate. Chang et al. [17] discuss mechanisms to improve the parallelism between reads and writes, and Kang et al. [50] discuss the use of SALP as a way of tolerating long write latencies to DRAM, which they identify as one of the three key scaling challenges for DRAM, amongst refresh and variable retention time. We refer the reader to these works for more information about these parallelization techniques.

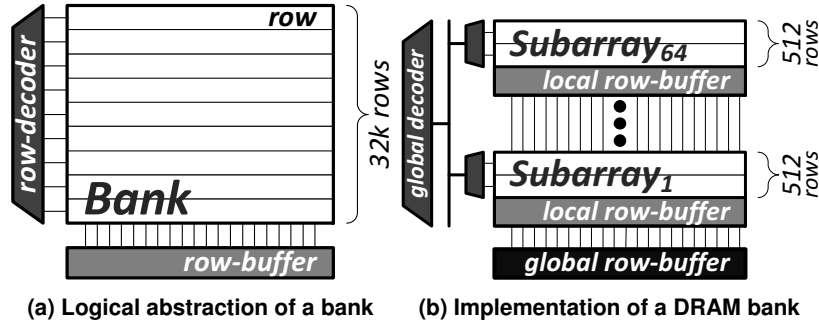


Fig. 6.2. DRAM Bank Organization. Reproduced from [57].

6.5.3 Reducing DRAM Latency and Energy

The DRAM industry has so far been primarily driven by the cost-per-bit metric: provide maximum capacity for a given cost. As shown in Figure 6.3, DRAM chip capacity has increased by approximately 16X in 12 years while the DRAM latency reduced by only approximately 20%. This is the result of a deliberate choice to maximize capacity of a DRAM chip while minimizing its cost. We believe this choice needs to be revisited in the presence of at least two key trends. First, DRAM latency is becoming more important especially for response-time critical workloads that require QoS guarantees [28]. Second, DRAM capacity is becoming very hard to scale and as a result manufacturers likely need to provide new values for the DRAM chips, leading to more incentives for the production of DRAMs that are optimized for objectives other than mainly capacity maximization.

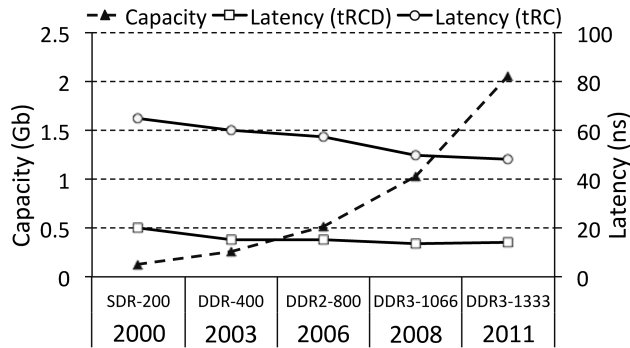


Fig. 6.3. DRAM Capacity & Latency Over Time. Reproduced from [68].

To mitigate the high area overhead of DRAM sensing structures, commodity DRAMs (shown in Figure 6.4a) connect many DRAM cells to each sense-amplifier through a wire called a bitline. These bitlines have a high parasitic capacitance due

to their long length, and this bitline capacitance is the dominant source of DRAM latency. Specialized low-latency DRAMs (shown in Figure 6.4b) use shorter bitlines with fewer cells, but have a higher cost-per-bit due to greater sense-amplifier area overhead. We have recently shown that we can architect a heterogeneous-latency bitline DRAM, called Tiered-Latency DRAM (TL-DRAM) [68], shown in Figure 6.4c, by dividing a long bitline into two shorter segments using an isolation transistor: a low-latency segment can be accessed with the latency and efficiency of a short-bitline DRAM (by turning off the isolation transistor that separates the two segments) while the high-latency segment enables high density, thereby reducing cost-per-bit. The additional area overhead of TL-DRAM is approximately 3% over commodity DRAM. Significant performance and energy improvements can be achieved by exposing the two segments to the memory controller and system software such that appropriate data is cached or allocated into the low-latency segment. We expect such approaches that design and exploit heterogeneity to enable/achieve the best of multiple worlds [84] in the memory system can lead to other novel mechanisms that can overcome difficult contradictory tradeoffs in design.

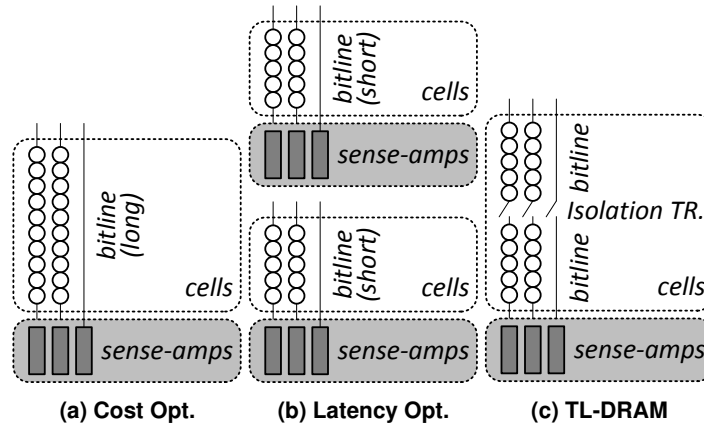


Fig. 6.4. Cost Optimized Commodity DRAM (a), Latency Optimized DRAM (b), Tiered-Latency DRAM (c). Reproduced from [68].

Another promising approach to reduce DRAM energy is the use of dynamic voltage and frequency scaling (DVFS) in main memory [27, 29]. David et al. [27] make the observation that at low memory bandwidth utilization, lowering memory frequency/voltage does not significantly alter memory access latency. Relatively recent works have shown that adjusting memory voltage and frequency based on predicted memory bandwidth utilization can provide significant energy savings on both real [27] and simulated [29] systems. Going forward, memory DVFS can enable dynamic heterogeneity in DRAM channels leading to new customization and optimization mechanisms. Also promising is the investigation of more fine-grained

power management methods within the DRAM rank and chips for both active and idle low power modes.

6.5.4 Exporting Bulk Data Operations to DRAM

Today's systems waste significant amount of energy, DRAM bandwidth and time (as well as valuable on-chip cache space) by sometimes unnecessarily moving data from main memory to processor caches. One example of such wastage sometimes occurs for bulk data copy and initialization operations in which a page is copied to another or initialized to a value. If the copied or initialized data is not immediately needed by the processor, performing such operations within DRAM (with relatively small changes to DRAM) can save significant amounts of energy, bandwidth, and time. We observe that a DRAM chip internally operates on bulk data at a row granularity. Exploiting this internal structure of DRAM can enable page copy and initialization to be performed entirely within DRAM without bringing any data off the DRAM chip, as we have shown in recent work [102]. If the source and destination page reside within the same DRAM subarray, our results show that a page copy can be accelerated by more than an order of magnitude (~ 11 times), leading to an energy reduction of ~ 74 times and *no* wastage of DRAM data bus bandwidth [102]. The key idea is to capture the contents of the source row in the sense amplifiers by activating the row, then *deactivating* the source row (using a new command which introduces very little hardware cost, amounting to less than 0.03% of DRAM chip area), and immediately activating the destination row, which causes the sense amplifiers to drive their contents into the destination row, effectively accomplishing the page copy (shown at a high level in 6.5). Doing so reduces the latency of a 4KB page copy operation from ~ 1000 ns to less than 100ns in an existing DRAM chip. Applications that have significant page copy and initialization lead to large system performance and energy efficiency improvements [102]. Future software can be designed in ways that can take advantage of such fast page copy and initialization operations, leading to benefits that may not be apparent in today's software that tends to minimize such operations due to their current high cost.

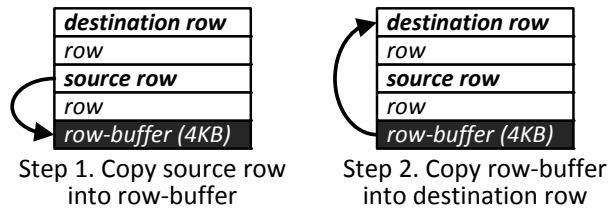


Fig. 6.5. High-level idea behind RowClone's in-DRAM page copy mechanism.

Going forward, we believe acceleration of other bulk data movement and computation operations in or very close to DRAM, via similar low-cost architectural support mechanisms, can enable promising savings in system energy, latency, and bandwidth. Given the trends and requirements described in Section 6.2, it is likely time to re-examine the partitioning of computation between processors and DRAM, treating memory as a first-class accelerator as an integral part of a heterogeneous parallel computing system [84].